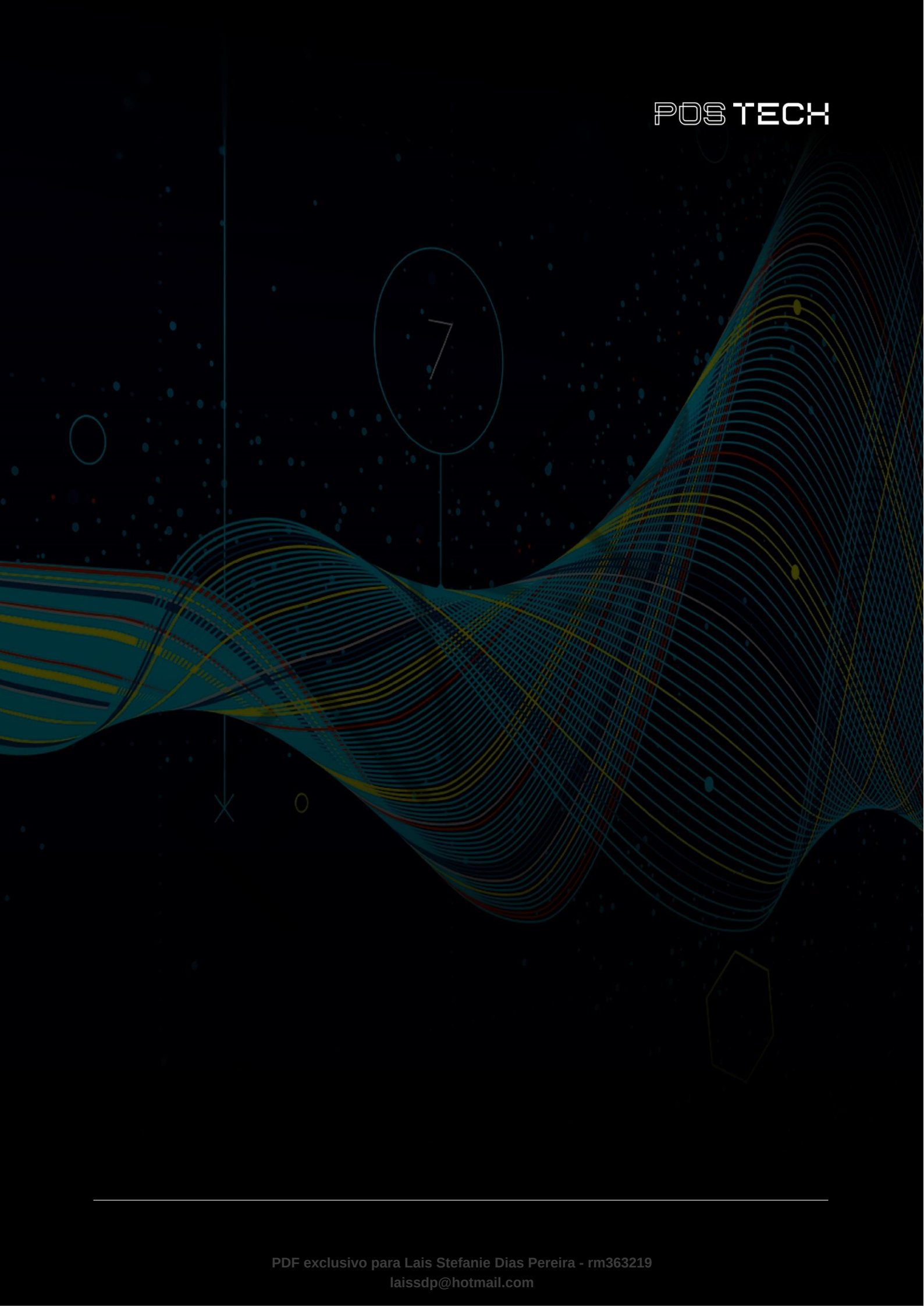




SUMÁRIO

O QUE VEM POR AÍ?	3
HANDS ON	4
SAIBA MAIS	5
MERCADO, CASES E TENDÊNCIAS	21
O QUE VOCÊ VIU NESTA AULA?	22
REFERÊNCIAS.....	23

O QUE VEM POR AÍ?

Agora teremos a oportunidade de construir uma aplicação completa de ML do zero, explorando desde o treinamento do modelo até a criação de uma interface interativa para usuários finais. Utilizaremos aqui tecnologias como o **Docker**, **Flask** e **Streamlit**. Este case abordará os aspectos técnicos de ML em produção, preparando você para criar soluções práticas e escaláveis no mundo real.



HANDS ON

Agora, vamos desenvolver uma aplicação completa de ML para prever a carga de aquecimento necessária para manter o conforto térmico em edifícios, utilizando o dataset **Energy Efficiency**. A solução combina as ferramentas **Docker**, **Flask** e **Streamlit**, seguindo uma arquitetura modular que inclui:

- **Treinamento do Modelo:** um container Docker responsável por carregar o dataset, treinar um modelo de regressão com **Random Forest** e otimizar hiperparâmetros usando **GridSearchCV**. O modelo treinado é salvo em um volume Docker para persistência;
- **API Flask:** um container que expõe um endpoint `/predict` para realizar inferências com o modelo treinado. A API valida os dados de entrada usando a biblioteca **Pydantic** e retorna previsões em formato JSON;
- **Interface Streamlit:** um container que fornece uma interface gráfica para inserir dados preditivos e consumir a API **Flask**, exibindo as previsões de carga de aquecimento de forma interativa.

A estrutura do projeto é organizada em pastas separadas para cada componente (treinamento, API e interface), com um arquivo `docker-compose.yml` para orquestrar os containers e um script `start.sh` para automatizar a execução. Ao final, os alunos e alunas poderão acessar a interface **Streamlit** via navegador, inserir dados e visualizar as previsões geradas pelo modelo.

SAIBA MAIS

Agora construiremos uma aplicação de ML para prever a carga de aquecimento (em kWh/m²) necessária para manter o conforto térmico em edifícios. utilizando o dataset [Energy Efficiency Dataset](#).

A previsão de eficiência energética tem se tornado uma área estratégica no mercado devido à crescente demanda por soluções sustentáveis e redução de custos operacionais. Modelos de ML, como o que você aprenderá nesta aula, são amplamente aplicados em setores como construção civil, arquitetura e energia renovável para projetar edifícios mais eficientes e com menor impacto ambiental.

Implementaremos aqui todas as ferramentas apresentadas até o momento em nossas aulas (Docker, Flask e Streamlit), conforme o desenho de arquitetura da Tabela a seguir.

Dados utilizados

O Energy Efficiency Dataset contém informações detalhadas sobre as variáveis e suas unidades de medida. A seguir, serão apresentadas as variáveis, suas descrições e respectivas unidades.

Variáveis preditoras

Variável	Descrição	Unidade
Relative Compactness	Relação entre o volume de construção e o espaço necessário para conter o mesmo volume.	Valor adimensional
Surface Area	Área total da superfície externa do edifício.	m ² (metros quadrados)
Wall Area	Área total das paredes externas do edifício.	m ² (metros quadrados)
Roof Area	Área total do telhado do edifício.	m ² (metros quadrados)
Overall Height	Altura total do edifício (do piso ao topo).	m (metros)

Orientation	Orientação do edifício em relação aos pontos cardeais (2 = Norte, 3 = Leste, etc.).	Inteiro
Glazing Area	Proporção da área envidraçada em relação à superfície total.	Valor adimensional
Glazing Area Distribution	Distribuição da área envidraçada nas fachadas (0 = nenhuma, 1-4 = distribuições específicas).	Inteiro

Tabela 1 - Variáveis preditoras
Fonte: elaborado pelo autor (2025)

A seguir, algumas informações importantes sobre os dados:

Orientation (orientação): é uma variável categórica codificada como inteiros:

- 2 = Norte;
- 3 = Leste;
- 4 = Sul;
- 5 = Oeste.

Glazing Area (área envidraçada): representa a fração da superfície total que é envidraçada, sem unidade específica.

Glazing Area Distribution (distribuição da área envidraçada): define como as janelas estão distribuídas:

- 0 = Sem janelas;
- 1-4 = Diferentes padrões de distribuição de janelas.

Variáveis target

Variável	Descrição	Unidade
HeatingLoad (Y1)	Carga de aquecimento necessária para manter o conforto térmico no edifício.	kWh/m ² (quilowatt-hora por metro quadrado)
CoolingLoad (Y2)	Carga de resfriamento necessária para manter o conforto térmico no edifício.	kWh/m ² (quilowatt-hora por metro quadrado)

Tabela 2 - Variáveis target
Fonte: Elaborado pelo autor (2025)

Ambas as variáveis alvo estão em **kWh/m²**, o que indica o consumo de energia por metro quadrado do edifício. No presente case, utilizaremos a variável Heating Load apenas.

Componentes da Solução

A solução (Tabela 1) será composta por:

1. **Volume Docker:** para persistir o modelo treinado;
2. **Container de Treinamento (Trainer):** realiza o treinamento de um modelo de regressão com Random Forest e otimiza seus hiperparâmetros;
3. **Container de API Flask (ML API):** oferece um endpoint `/predict` para realizar inferências;
4. **Container de Interface Streamlit (UI):** interface gráfica para consumir a API e realizar previsões.

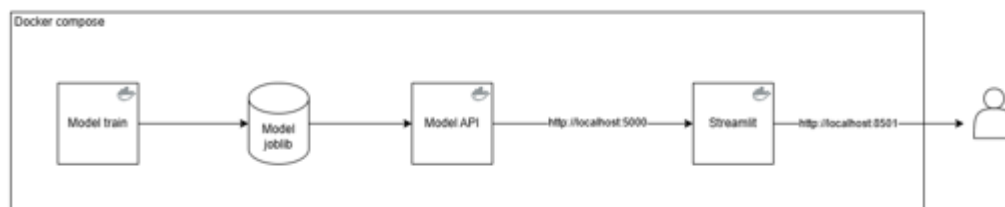


Figura 1 - Desenho de arquitetura da solução
Fonte: elaborado pelo autor (2025)

O repositório terá a seguinte estrutura:

```
```bash

ml-docker-app/

|

├── train/

| ├── train.py # Script de treinamento
| ├── requirements.txt # Dependências do container de treinamento
| └── Dockerfile # Dockerfile do treinamento
|
|
├── api/

| ├── app.py # API Flask
| ├── requirements.txt # Dependências do container da API
| └── Dockerfile # Dockerfile da API
|
|
├── streamlit/

| ├── app.py # Interface Streamlit
| ├── requirements.txt # Dependências do container do Streamlit
| └── Dockerfile # Dockerfile do Streamlit
|
|
└── docker-compose.yml # Orquestração dos serviços
```



```
|— start.sh # Script para execução automatizada
|— README.md # Documentação
...

```

A primeira componente de treinamento do modelo o container de treinamento realiza:

- Carregamento do dataset `load_diabetes`;
- Treinamento de um modelo com a classe `RandomForestRegressor` e otimização de hiperparâmetros usando `GridSearchCV`;
- Salvamento do modelo treinado em um volume Docker compartilhado.

Código do `train/train.py`:

```
```python
import pandas as pd

from sklearn.model_selection import train_test_split,
GridSearchCV

from sklearn.ensemble import RandomForestRegressor
from sklearn.pipeline import Pipeline
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import mean_absolute_error

import pickle

import os

# Carregar o dataset Energy Efficiency

```

```
url = "https://archive.ics.uci.edu/ml/machine-learning-
databases/00242/ENB2012_data.xlsx"

data = pd.read_excel(url)

# Separar as variáveis preditoras (X) e as variáveis alvo (y)
X = data.iloc[:, :-2] # Todas as colunas exceto as duas
últimas (Heating&CoolingLoad)
y = data["Y1"] # Carga de aquecimento ("HeatingLoad")

# Dividir os dados em treino e teste
X_train, X_test, y_train, y_test = train_test_split(X, y,
test_size=0.2, random_state=42)

# Criar o pipeline
pipeline = Pipeline([
    ("imputer", SimpleImputer(strategy="mean")), # Tratamento
de valores ausentes
    ("scaler", StandardScaler()), # Escalonamento
    ("model", RandomForestRegressor(random_state=42)) #
Modelo
])

# Otimização de hiperparâmetros
param_grid = {
    "model__n_estimators": [50, 100, 200],
```

```
"model__max_depth": [None, 10, 20],
    "model__min_samples_split": [2, 5, 10],
    "model__min_samples_leaf": [1, 2, 4],
}

grid_search = GridSearchCV(
    estimator=pipeline,
    param_grid=param_grid,
    scoring="neg_mean_absolute_error",
    cv=3,
    verbose=2,
    n_jobs=-1
)

# Treinar e encontrar os melhores parâmetros
grid_search.fit(X_train, y_train)
print("Melhores Hiperparâmetros:", grid_search.best_params_)

# Avaliar no conjunto de teste
best_model = grid_search.best_estimator_
predictions = best_model.predict(X_test)
print("Erro Médio Absoluto (MAE):", mean_absolute_error(y_test,
predictions))
```

```
# Salvar o modelo treinado

model_path = '/model_data/pipeline.pkl'

os.makedirs(os.path.dirname(model_path), exist_ok=True)

with open(model_path, 'wb') as f:
    pickle.dump(best_model, f)

print(f"Modelo salvo em {model_path}")

...
```

Dependências train/requirements.txt:

```
...

numpy==1.24.3
openpyxl==3.1.2
pandas==1.5.3
scikit-learn==1.2.2
...
```

Dockerfile train/Dockerfile:

```
```Dockerfile

FROM python:3.9-slim

WORKDIR /app

COPY requirements.txt .

RUN pip install -r requirements.txt
```

```
COPY . .

CMD ["python", "train.py"]

...
```

Agora, seguimos com os insumos para a componente da API de inferência com o modelo treinado utilizando Flask.

A API Flask oferece aqui:

- Validação de dados de entrada usando a biblioteca Pydantic;
- Endpoint `/predict` para inferência no modelo treinado.

Código de `api/app.py`:

```
```python  
from flask import Flask, request, jsonify, Response  
from pydantic import BaseModel, ValidationError  
import pickle  
  
app = Flask(__name__)  
  
# Carregar o pipeline do modelo treinado  
pipeline_path = '/model_data/pipeline.pkl'  
with open(pipeline_path, 'rb') as f:  
    pipeline = pickle.load(f)  
  
# Classe para validar as entradas  
class InputData(BaseModel):
```

```
RelativeCompactness: float

SurfaceArea: float

WallArea: float

RoofArea: float

OverallHeight: float

    Orientation: int

GlazingArea: float

GlazingAreaDistribution: int


@app.route('/predict', methods=['POST'])
def predict():
    try:
        input_data = InputData(**request.get_json())

        features = [
            input_data.RelativeCompactness, input_data.SurfaceArea,
            input_data.WallArea, input_data.RoofArea,
            input_data.OverallHeight,
            input_data.Orientation, input_data.GlazingArea,
            input_data.GlazingAreaDistribution
        ]

        prediction = pipeline.predict(features)

        status_code = 200

        status_message = Response(status=status_code).status

        return jsonify({
```

```
        "status": status_message,
        "data": {"prediction": prediction[0]}
    )), status_code

    except ValidationError as e:
status_code = 400

        return jsonify({"error": e.errors()}), status_code

if __name__ == '__main__':
app.run(host='0.0.0.0', port=5000)
...
```

Dependências do container da API `api/requirements.txt`:

```
...
Flask==2.3.2
pydantic==1.10.2
scikit-learn==1.2.2
numpy==1.24.3
...
```

A seguir, serão fornecidos os insumos para a criação da interface com a biblioteca Streamlit.

O Streamlit permite:

- Inserir dados preditivos não normalizados;

- Fazer previsões consumindo o endpoint da API.

Código do streamlit/app.py:

```
```python
import streamlit as st
import requests

st.title("Previsão de Carga de Aquecimento")

st.write("Insira os valores das variáveis preditoras com base nas características do edifício:")

Campos de entrada com unidades de medida nas descrições
RelativeCompactness = st.number_input("Compacidade Relativa (adimensional)", value=0.75)

SurfaceArea = st.number_input("Área da Superfície (m²)", value=500.0)

WallArea = st.number_input("Área da Parede (m²)", value=300.0)

RoofArea = st.number_input("Área do Telhado (m²)", value=150.0)

OverallHeight = st.number_input("Altura Total (m)", value=3.0)

Orientation = st.selectbox("Orientação (2=Norte, 3=Leste, 4=Sul, 5=Oeste)", options=[2, 3, 4, 5])

GlazingArea = st.number_input("Área de Vidro (adimensional)", value=0.1)

GlazingAreaDistribution = st.selectbox("Distribuição da Área de Vidro (0=Sem, 1-4=Distribuições)", options=[0, 1, 2, 3, 4])
```



```
Botão para realizar a predição

ifst.button("Prever"):

 # Preparar os dados para envio à API
 input_data = {

 "RelativeCompactness": RelativeCompactness,

 "SurfaceArea": SurfaceArea,

 "WallArea": WallArea,

 "RoofArea": RoofArea,

 "OverallHeight": OverallHeight,

 "Orientation": Orientation,

 "GlazingArea": GlazingArea,

 "GlazingAreaDistribution": GlazingAreaDistribution

 }

 response = requests.post("http://api:5000/predict",
 json=input_data)

 # Mostrar o resultado

 if response.status_code == 200:

 result = response.json()

 st.success(f"Previsão da Carga de Aquecimento:
 {result['data']['prediction']:.2f} kWh/m²")

 else:

 st.error("Erro ao realizar a predição. Verifique os valores de
 entrada.")
```

```
...
```

Agora será fornecido o arquivo para orquestração dos containers da solução com Docker Compose. Arquivo `docker-compose.yml`:

```
```yaml
version: '3.9'

services:
  trainer:
    build: ./train
    volumes:
      - model_volume:/model_data
    command: ["python", "train.py"]
    restart: "no"
  api:
    build: ./api
    ports:
      - "5000:5000"
    volumes:
      - model_volume:/model_data
  depends_on:
    - trainer
```

```
streamlit:
    build: ./streamlit
    ports:
        - "8501:8501"
depends_on:
- api

volumes:
model_volume:
    ...
```

Para criar os containers de forma ordenada, vamos usar um arquivo `start.sh` com os seguintes comandos para execução:

```
```bash
#!/bin/bash

echo "Treinando o modelo..."
docker-compose run trainer

echo "Iniciando a API..."
docker-compose up -d api

echo "Verificando API..."
until curl -s http://localhost:5000/ > /dev/null; do
```

```
echo "Aguardando API..."

sleep 2

done

echo "Iniciando Streamlit..."

docker-compose up -d streamlit

` ``
```

Execute o script no terminal:

```
` ``bash

bash start.sh

` ``
```

Acesse a URL <http://localhost:8501> para a interface do Streamlit, e você visualizará a seguinte tela:



**Previsão de Carga de Aquecimento**

Insira os valores das variáveis preditoras com base nas características do edifício:

Compartimento Potência (dimensional)

0,75

Área da Superfície (m²)

500,00

Área da Parede (m²)

500,00

Área do Teto (m²)

150,00

Altura Total (m)

3,00

Orientação (2=Nordeste, 3=Leste, 4=Sul, 5=Oeste)

4

Área de Vidro (dimensional)

0,10

Distribuição da Área de Vidro (2=0, 3=1, 4=Distribuição)

3

Previsão

Previsão da Carga de Aquecimento: 19.20 kWh/m²

Figura 2 – Tela Streamlit  
Fonte: elaborado pelo autor (2025)

Para deletar os containers, imagens e volume criados, execute no terminal:

```
docker-compose down --rmall --volumes --remove-orphans.
```

## MERCADO, CASES E TENDÊNCIAS

A aplicação de ML em produção tem revolucionado diversas indústrias, permitindo a criação de soluções automatizadas para problemas complexos em larga escala. Modelos de ML não apenas geram insights poderosos, mas também integram sistemas que impactam diretamente a tomada de decisão em diversas empresas.

A [Netflix](#), por exemplo, elaborou um sistema para detecção de fraude para seu serviço de streaming. No caso, utilizam modelos avançados de ML para identificar e mitigar atividades fraudulentas em sua plataforma, como compartilhamento de contas e acessos não autorizados. Usando algoritmos como redes neurais e métodos de aprendizado supervisionado, o sistema analisa padrões de comportamento para detectar anomalias em tempo real, protegendo tanto a empresa quanto os usuários.

## O QUE VOCÊ VIU NESTA AULA?

Nesta aula você explorou como construir uma aplicação completa de ML para resolver um problema real utilizando o Energy Efficiency Dataset. Desde o treinamento do modelo até a criação de uma interface interativa, você aprendeu a usar tecnologias modernas, como Docker, Flask, e Streamlit, para encapsular e disponibilizar soluções de aprendizado de máquina em produção. No treinamento, você utilizou um pipeline robusto com Random Forest, otimizando hiperparâmetros para maximizar a precisão do modelo na previsão da carga de aquecimento de edifícios.

Além disso, você implementou uma API em Flask para realizar inferências de forma escalável e uma interface Streamlit amigável para permitir interações simples e intuitivas. Durante o processo, você também compreendeu como manipular dados reais com diferentes unidades de medida e como transformar essas informações em insights acionáveis. Essa experiência prática reforça não apenas os fundamentos de ML, mas também as boas práticas de engenharia para levar modelos ao ambiente de produção.

## REFERÊNCIAS

ARCHIVE.ICS.UCI. **Energy Efficiency Dataset**. Disponível em: <<https://archive.ics.uci.edu/dataset/242/energy+efficiency>>. Acesso em: 17 jan. 2025.

DOCKER. **Docker Documentation**. Disponível em: <<https://docs.docker.com/>>. Acesso em: 17 jan. 2025.

FLASK. **Flask Documentation**. Disponível em: <<https://flask.palletsprojects.com/en/stable/>>. Acesso em: 17 jan. 2025.

SCIKIT-LEARN. **Scikit-Learn 0.21 Documentation**. Disponível em: <<https://scikit-learn.org/0.21/documentation.html>>. Acesso em: 17 jan. 2025.

STREAMLIT. **Streamlit Documentation**. Disponível em: <<https://docs.streamlit.io/>>. Acesso em: 17 jan. 2025.

## **PALAVRAS-CHAVE**

ML pipeline, Docker, Flask, Streamlit.

EMEND



The background is a dark blue field filled with numerous small, light blue dots. Overlaid on this are several large, wavy, translucent lines in shades of blue and yellow. A vertical line on the left side has a small 'x' at its base. A circle containing the number '7' is positioned in the upper left. A hexagon is located in the lower right. The text 'POSTECH' is centered in a white, outlined font.

POSTECH